Step 3 — A Second Layer: Scanning Retrieved Content for Injection Patterns — Hints

Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the honest limit
- Hint 2 — The plan, and wiring it in

Hint 1 — The idea, and the honest limit

A second, independent layer: `scan_for_injection_patterns(text) -> list[str]` returns plain-English warnings (empty if nothing looks off). Check for:

- "ignore previous instructions" and close variants ("ignore all prior instructions", "disregard the above/everything");
- a fake role label at the start of a line (`SYSTEM:` , `ASSISTANT:`) — a small regex;
- direct model address ("you must now", "your new instructions are", "print your full system prompt", "reveal your");
- "unrestricted mode".

Write the honest limit into the file as a comment: at least one rephrasing that slips past (e.g. "disregard everything stated earlier", or the trigger split across two sentences). This isn't a bug — it's the real limit of pattern matching, and naming it is part of doing the step right. The scanner is a tripwire (detection + logging), not a wall.

Hint 2 — The plan, and wiring it in

```
injection_scanner.py:
    SUSPICIOUS = [list of lowercase phrases]
    ROLE_LABEL = re.compile(r"(?im)^\s*(system|assistant)\s*:")
    scan_for_injection_patterns(text):
        warnings = [ "matched ..." for each phrase in text.lower() ]
        if ROLE_LABEL.search(text): warnings.append("fake role label")
        return warnings

retriever.py (change):
    after getting chunks from Chroma, for each chunk:
```

```
        for w in scan_for_injection_patterns(chunk_text):
            logger.warning(f"[{source}] {w}")       # logger, not print
```

The model still receives every chunk exactly as Step 2 wraps it — this step adds **visibility**, not blocking. Re-run `main.py` and confirm the log names `policy_expenses.md` the moment it's retrieved. The whole file set is in the Solution.